

CSCE 669 COMPUTATIONAL OPTIMIZATION · MODULE 02 · LECTURE NOTES

Convexity

The property everything else depends on.

MODULE OUTCOMES

- Define convex sets and convex functions.
- Prove convexity by the standard tests.
- Apply the operations that preserve convexity.
- Recognise the standard convex functions.
- Explain disciplined convex programming.

1 The definitions

$$\mathbf{C} \text{ convex} \Leftrightarrow \theta \mathbf{x} + (1-\theta)\mathbf{y} \in \mathbf{C} \qquad f \text{ convex} \Leftrightarrow f(\theta \mathbf{x} + (1-\theta)\mathbf{y}) \leq \theta f(\mathbf{x}) + (1-\theta)f(\mathbf{y})$$

A set is convex when the entire segment joining any two of its points lies inside it. A function is convex when the chord between any two points on its graph lies above the function. **And the two are the same definition:** f is convex exactly when its **epigraph** — the set of points lying on or above its graph, $\{(\mathbf{x}, t) : f(\mathbf{x}) \leq t\}$ — is a convex set. **The epigraph formulation unifies them and makes several later results immediate rather than separate**, which is why it is worth carrying rather than treating the two definitions as a coincidence.

Test	Condition	Use when
Zeroth order	The definition, applied directly.	Small cases, and functions that are not differentiable — where the other tests do not apply.
First order	$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^T(\mathbf{y} - \mathbf{x})$ for all $\mathbf{x}, \mathbf{y}$.	The tangent plane at any point lies below the function everywhere. Conceptually the most important of the three — a global underestimator from purely local information, which is exactly what Module 01 §3 said was otherwise impossible, and is where duality originates.
Second order	The Hessian $\nabla^2 f$ is positive semidefinite everywhere.	Twice-differentiable functions, and the usual working test. Checking positive semidefiniteness is itself work in high dimensions.
By construction	Built from known convex pieces using operations that preserve convexity.	Almost always the practical route — §2. Proving convexity from the definition is rare in practice.

2 The convexity calculus

```

PRESERVE CONVEXITY
  non-negative weighted sum    sum a_i f_i,  a_i >= 0
  affine composition           f(Ax + b)
  POINTWISE MAXIMUM            max(f_1, ..., f_k)
    -- even of INFINITELY many. The most useful rule here.
  partial minimisation         inf_y f(x,y), f jointly convex
  perspective                   t * f(x/t),  t > 0
  composition                   h(g(x)) if h convex AND NON-DECREASING

DO NOT PRESERVE IT
  pointwise MINIMUM            min(f_1, f_2)
  product                      f * g
  difference                   f - g

THE COMPOSITION RULE IS WHERE MISTAKES HAPPEN
  exp(convex)    convex    -- exp is convex and increasing
  log(concave)   concave   -- log is concave and increasing
  sqrt(convex)   NOT       -- sqrt is increasing but CONCAVE
  (convex)^2     NOT       -- unless the inner f is >= 0

```

That pointwise maximum preserves convexity while pointwise minimum does not is the most useful asymmetry in the calculus. It is why a maximum of affine functions is convex — which makes hinge loss convex, makes the dual function concave (Module 04), and underlies piecewise-linear modelling generally. **And it is why 'the best of several options' is usually *not* convex**, which is the formal reason discrete choice makes problems hard.

DISCIPLINED CONVEX PROGRAMMING AUTOMATES THIS

CVXPY and its relatives refuse any expression they cannot verify is convex, by applying the rules of §2 mechanically to a library of known atoms with known curvature and monotonicity.

So `cp.sqrt(cp.sum_squares(x) + 1)` may be rejected while `cp.norm(cp.hstack([x, 1]))` is accepted, even though they are mathematically identical — the second is recognisably a norm and the first is a square root of a convex function, which the rules correctly decline to certify.

That is frustrating the first time and it is exactly the point. If the modelling language cannot verify convexity, neither can the solver, and the polynomial-time guarantee the solver offers is contingent on the problem actually being convex. A solver handed a non-convex problem returns a number with no warranty.

Learning to write a problem in a form DCP accepts is learning the convexity calculus, enforced by a compiler. It is the fastest way to internalise §2, and it is a rare case of a tool teaching the theory as a side effect of being strict.

3 The standard atoms

Function	Curvature	Note
Any norm $\ \mathbf{x}\ $	Convex	Directly from the triangle inequality and homogeneity. $\ \cdot\ _1$ gives sparsity, $\ \cdot\ _2$ gives shrinkage (CSCE 633 Module 04 §1) — and the geometric argument there is this convexity, seen as a constraint set.
$\mathbf{x}^T \mathbf{P} \mathbf{x}$ with $\mathbf{P}$ positive semidefinite	Convex	Quadratic forms. Least squares is one; so is the SVM's objective.
$\exp(\mathbf{ax})$, $\mathbf{x} \log \mathbf{x}$	Convex	And entropy $-\sum \mathbf{p} \log \mathbf{p}$ is concave , which is why maximum-entropy problems are tractable.
log-sum-exp	Convex	A smooth approximation to the maximum , and the normalising term of the softmax (CSCE 633 Module 02 §3) — so cross-entropy's convexity in the logits comes from here.
max of affine functions	Convex	Hinge loss is exactly this (CSCE 633 Module 07), which is why the SVM is a convex problem.
log det $\mathbf{X}$ over positive definite $\mathbf{X}$	Concave	Appears throughout semidefinite programming and in maximum-likelihood covariance estimation.
Indicator of a convex set	Convex	Zero inside, $+\infty$ outside. This is how constraints are folded into an objective , which unifies constrained and unconstrained formulations.

Most of the losses used in the previous two courses are on this list — squared error, hinge, logistic, and every norm-based regulariser. **Which retroactively explains why those problems were well behaved**, and why neural networks (CSCE 636), which compose these atoms non-monotonically through many layers, are not.

4 What convexity buys

- **Every local minimum is a global minimum. The reason local methods work at all** (Module 01 §3), and the root of every other item on this list.
- **The tangent is a global underestimator** (the first-order condition), **so lower bounds come for free** — and a bound is a certificate. This is where Module 04's duality originates and why a dual solution *proves* optimality rather than merely suggesting it.
- **Strong duality holds under mild regularity conditions** (Module 04 §2), so the primal and dual optimal values coincide and either certifies the other.
- **Convergence rates are provable**, and they depend on measurable properties of the problem — the condition number — rather than on the starting point or on luck (Module 05).
- **And the feasible set is connected**, so any feasible point can be reached from any other without leaving the set, which is what makes interior point methods (Module 07) possible at all.

WHEN IT IS NOT CONVEX, CONVEXIFY

Change variables. Geometric programs — which look thoroughly non-convex — become convex under a logarithmic substitution, and that single transformation covers a large class of engineering design problems in circuits, structures, and chemical processes. **Looking for a change of variables is the first thing to try** and is frequently forgotten.

Relax. Replace an integer constraint $x \in \{0,1\}$ with the interval $0 \leq x \leq 1$ (Module 09), or a rank constraint with a nuclear norm (Module 10). **The relaxation is convex, its optimum bounds the true one, and the gap is measurable.**

Restrict. Optimise over a convex subset of the feasible region — accepting a possibly worse answer in exchange for a guarantee about it, which is frequently the better trade.

Or accept local optimality and say so. That is CSCE 636's entire position, and it is a perfectly legitimate engineering stance **when it is stated rather than assumed** — the failure is claiming a global optimum from a local method, not using one.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Boyd & Vandenberghe — Convex Optimization, chapters 2 and 3 (free)	The definitive treatment of §1 through §3 , including the full atom library and every preservation rule with proof.
Grant, Boyd & Ye — Disciplined Convex Programming (free)	The §2 callout's ruleset , stated formally — which explains exactly why CVXPY rejects what it rejects.
CVXPY — the atomic function library and DCP tutorial (free)	The atoms of §3 with their curvature and monotonicity, machine-readable and worth browsing.
Boyd — EE364A lectures 2 and 3 (free video)	The same material lectured, with the geometric intuition that the book states more tersely.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Prove three functions convex by three different tests** — definition, first order, second order.
- 2 Show that sqrt of a convex function need not be convex** with an explicit counterexample.
- 3 Show the pointwise maximum of two convex functions is convex**, and that the minimum need not be.
- 4 Prove that every norm is convex** from the triangle inequality.

- 5 Verify that log-sum-exp is convex by computing its Hessian.
- 6 **Write ten expressions in CVXPY**, five it accepts and five it rejects, and explain each rejection.
- 7 **Rewrite a rejected expression** into an equivalent accepted one.
- 8 Take three loss functions from CSCE 633 and classify each as convex or not.
- 9 **Convert a geometric program to convex form** by logarithmic change of variables.
- 10 Take a non-convex problem you care about and apply each of §4's four moves.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define convex sets and convex functions, and relate them via the epigraph.
- 2 Give three tests for convexity and say when each applies.
- 3 Why is the first-order condition conceptually important?
- 4 Name six operations that preserve convexity and three that do not.
- 5 State the composition rule and give a case where it fails.
- 6 Why is pointwise maximum convex and pointwise minimum not?
- 7 Name seven standard convex atoms.
- 8 Give five consequences of convexity.
- 9 Give four ways to handle a non-convex problem.